DookuG modul dokumentáció

Table of Contents

1. Technológiák . . . 2
2. Tartalom . . . 2
3. DookuG modul kliens . . . 2
   3.1. Általános . . . 3
   3.2. Konfiguráció . . . 3
      3.2.1. A következő értékek megadása célszerű a kliens működéséhez: . . . 3
   3.3. Kliens használata . . . 4
      3.3.1. Kliens metódusok . . . 4
      3.3.2. Saxon(-HE) engine használata a kliensben . . . 10
      3.3.3. Hibakezelés . . . 11
4. DOCUMENT service . . . 11
   4.1. Dokumentum generálása . . . 11
      4.1.1. Dokumentum generálása request body alapján . . . 11
      4.1.2. Dokumentum generálása multipart form alapján . . . 12
      4.1.3. Dokumentum generálása mentett sablon alapján . . . 13
   4.2. Dokumentum metaadatainak lekérdezése . . . 15
   4.3. Dokumentum lekérdezése . . . 17
   4.4. Template cache . . . 17
   4.5. Saját betűtípusok használata . . . 17
   4.6. PDF-ek digitális aláírása . . . 20
      4.6.1. Konfiguráció . . . 20
5. Telepítés, üzembe helyezés . . . 21
   5.1. DookuG Service Konfiguráció . . . 21
      5.1.1. Kubernetes deployment . . . 22
      5.1.2. Health - startup/liveness/readiness . . . 22
6. Kiegészítő információk . . . 24
   6.1. Helpers - Template fájlokban használható helperek . . . 24
      6.1.1. Built-in helperek . . . 24
      6.1.2. Custom helperek . . . 24
7. Release notes . . . 29
   7.1. v0.4.0 . . . 29
   7.2. v0.5.0 . . . 30
      7.2.1. Changes / updates . . . 30
      7.2.2. Migration . . . 30
   7.3. v0.6.0 . . . 30
      7.3.1. Changes / updates . . . 30

7.4. v1.0.0 30
7.4.1. Changes / updates 30

Version: 1.0.0-SNAPSHOT

A projekt célja hogy általános template alapú dokumentum generálást tegyen lehetővé a kliensei számára.

A projekt microservice alapon tartalmaz minta service-ket:

- REST Service - Csak REST szükségleteket szolgál ki.

1. Technológiák

A technológiák melyeken alapszik:

Service (jar, war)

- Java 17+
- Maven 3.8.0+
- Java EE 10
- Wildfly 27.0.1Final

2. Tartalom

Projekt struktúra

- `/docs` - Dokumentációk tárhelye
- `/etc` - Konfigurációk, fejlesztési környezet scriptek, stb.
- `/dookug-bom` - Maven dependency definíciók
- `/dookug-api` - modul API
- `/dookug-common` - Közös, általános kódok melyeket a service-k használnak
- `/dookug-client` - Kliens a modulhoz
- `/dookug-engine` - Feldolgozást segítő tool implementációk
- `/dookug-document` - Dokumentum generálást összefogó REST szolgáltatások
- `/dookug-testsuite` - Fejlesztői tesztesetek gyűjtő könyvtára

3. DookuG modul kliens

A kliens célja a DookuG modul kezelésének támogatása, amely különböző kiszolgáló metódusokat tartalmaz.

3.1. Általános

Használt technológiák:

- Java 11
- CDI 2.0
- commons-lang3 3.9
- Microprofile restclient 2.0
- Microprofile config 2.0
- Bouncycastle 1.61 (wildfly provided)

3.2. Konfiguráció

Microprofile Config segítségével tudjuk a kliens működését befolyásolni.

```
dookug:
    client:
        document/mp-rest/url: http://localhost:8082
        document/mp-rest/connectTimeout: 5000 #millisec
        document/mp-rest/readTimeout: 60000 #millisec
```

3.2.1. A következő értékek megadása célszerű a kliens működéséhez:

Paraméter kulcs	Kötelező	Leírás
dookug.client.document/mp-rest/url	igen	a dookug module elérési útvonala (base url)
dookug.client.document/mp-rest/connectTimeout		Connection Timeout megadása (ha coffee-t használunk, ez alapértelmezetten 5 másodperc)
dookug.client.document/mp-rest/readTimeout		Read Timeout megadása (ha coffee-t használunk, ez alapértelmezetten 1 perc)
dookug.service.cache.template.ttl	nem	Template cache lejárati ideje percben, alapértelmezetten 60 perc
dookug.service.cache.template.enablestatistic	nem	generálódjanak metrikák, alapértelmezetten nem generálódnak

További paraméterek megadása is lehetséges, lásd a Microprofile RestClient dokumentációban.

3.3. Kliens használata

A kliens használata mindössze egy pom.xml bejegyzés,

```
<dependency>
   <groupId>hu.icellmobilsoft.dookug</groupId>
   <artifactId>dookug-client</artifactId>
   <version>${version.dookug.client}</version>
</dependency>
```

majd egy @Inject,

```
@Inject
private DookugClient dookugClient;
```

mely az összes végponthoz definiál kliens hívást, és használhatjuk is az általa kínált metódusokat. Microprofile rest klienst használ az API híváshoz így a beállítások a már megszokott microprofile-config segítségével végezhetőek el. (lásd később)

WARNING	A kliens alkalmazás feladata, hogy a konténerbe mountolja be a `/home/icellmobilsoft/resources` mount point alá az általa használt resource-okat, melyekre a template-ekben hivatkozik!

3.3.1. Kliens metódusok

A kliensnek jelenleg több metódusa is van, melyekkel dokumentumot lehet generáltatni, a létrehozott dokumentumok metaadatait lehet lekérdezni, illetve magát a generált dokumentumot lehet lekérdezni.

A dokumentum generálása során a metódusok egy részénél a generált dokumentumot kapjuk vissza:

```
public GeneratedDocumentDto postDocumentGenerateEntityBody(Collection<TemplateType>
templates, Collection<ParameterType> parameters) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateEntityBody(Collection<TemplateType>
templates, ParametersDataType parametersData) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateEntityBody(Collection<TemplateType>
templates) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateMultipart(InputStream template,
Collection<ParameterType> parameters) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateMultipart(InputStream template,
ParametersDataType parametersData) throws BaseException;
```

```
public GeneratedDocumentDto postDocumentGenerateMultipart(InputStream template) throws
BaseException;

public GeneratedDocumentDto postDatabaseStoredTemplateDocumentGenerate(String
templateName, OffsetDateTime templateValidity, Collection<ParameterType> parameters)
throws BaseException;

public GeneratedDocumentDto postDatabaseStoredTemplateDocumentGenerate(String
templateName, OffsetDateTime templateValidity, ParametersDataType parametersData)
throws BaseException;
```

Fontos kiegészítés, hogy a `ParametersDataType` típusú paramétereket bár kézzel is létrehozhatjuk, de készült hozzájuk egy ParametersDataBuilder segédosztály, amiben fluent API segítségével állíthatjuk be a kívánt konfigurációkat. Létrehozását a SAXON template motor kívánta meg, mert az XSLT template-ek generálásához szükséges megadni további paramétereket is, melyek nélkül nem generálható dokumentum. A builderből SAXON engine-hez nagyon könnyen kinyerhető a kívánt konfiguráció a getSaxonParameters(byte[], byte[], boolean) metódus meghívásával, melynek a FOP konfiguráció tartalma, az XML dataset és az XML tömörítettségét jelző értéket lehet megadni, vagy az overload-olt metódusokon keresztül csak az XML fájl tartalmát.

```
GeneratedDocumentDto resp = client.postDatabaseStoredTemplateDocumentGenerate(
        TemplateLanguageType.HU, ①
        ParametersDataBuilder.getSaxonParameters(

FileUtil.readFileFromResource(DocumentServiceTestConstant.XSLT_TEMPLATE_PARAMS).getByt
es(StandardCharsets.UTF_8))); ②
```

① A template nyelve

② Az XML dataset tartalma (jelen esetben tömörítetlenül)

Ezek a metódusok - sikeres hívás esetén - egy GeneratedDocumentDTO-val térnek vissza, mely tartalmazza a generált fájlnevet, a generált objektumot, mint stream-et és a HTTP status kódot.

Paraméterként meg kell kapniuk a template listát, valamint a paramétereket, ha a template-ekben változók vannak, amiket ki kell cserélni értékekre, pl. HANDLEBARS template engine esetén. A paraméterek lehetnek key-value párok, vagy egy JSON objektum, mely tulajdonképp ezeket a key-value párokat tartalmazza.

A metódusok elkülönülnek a dokumentum generáláshoz felhasznált template megadásának formájában. A `postDocumentGenerateEntityBody()` metódusoknál a request body-ban küldjük a sablonként használt struktúrát, a `postDocumentGenerateMultipart()` metódusoknál form paraméterként, InputStream-ként, adható meg a sablon, míg a `postDatabaseStoredTemplateDocumentGenerate()` metódusoknál csak a sablon nevét kell megadni, mivel ezt a megadott tárolási metódusnak megfelelően kezeli a végpont.

Ezeken a metódusokon kívül lehetőség van még olyan dokumentum generálásra, ahol nem magát az elkészült fájlt kapjuk meg, hanem az azt leíró metaadatokat:

```
public DocumentMetadataResponse
postDocumentGenerateEntityBodyMetadata(Collection<TemplateType> templates,
Collection<ParameterType> parameters) throws BaseException;

public DocumentMetadataResponse
postDocumentGenerateEntityBodyMetadata(Collection<TemplateType> templates,
ParametersDataType parametersData) throws BaseException;

public DocumentMetadataResponse
postDocumentGenerateEntityBodyMetadata(Collection<TemplateType> templates) throws
BaseException;

public DocumentMetadataResponse postDocumentGenerateMultipartMetadata(InputStream
template, Collection<ParameterType> parameters) throws BaseException;

public DocumentMetadataResponse postDocumentGenerateMultipartMetadata(InputStream
template, ParametersDataType parametersData) throws BaseException;

public DocumentMetadataResponse postDocumentGenerateMultipartMetadata(InputStream
template) throws BaseException;

public DocumentMetadataResponse
postDatabaseStoredTemplateDocumentGenerateMetadata(String templateName, OffsetDateTime
templateValidity, Collection<ParameterType> parameters) throws BaseException;

public DocumentMetadataResponse
postDatabaseStoredTemplateDocumentGenerateMetadata(String templateName, OffsetDateTime
templateValidity, ParametersDataType parametersData) throws BaseException;

public DocumentMetadataResponse postStoredTemplateDocumentGenerateMetadata(String
templateName, OffsetDateTime templateValidity, TemplateStorageMethodType
templateStorageMethodType, Collection<ParameterType> parameters, ParametersDataType
parametersData) throws BaseException;
```

Ezek a metódusok - sikeres hívás esetén - egy DocumentMetadataResponse-val térnek vissza, mely tartalmazza a dokumentum egyedi azonosítóját, fájlnevét, tárolási módját, formátumát és a státuszát.

Hasonlóan, az előzőleg felsorolt metódusoknál, a sablon megadásának formájában ugyanúgy különülnek el a kliens hívások.

Ahhoz, hogy láthassuk, hogy már milyen fájlokat generáltunk lehetőség van a dokumentumok metaadatainak lekérdezésére:

```
public DocumentMetadataQueryResponse
postDocumentMetadataQuery(DocumentMetadataQueryParamsType queryParams) throws
BaseException;

public DocumentMetadataQueryResponse
```

```
postDocumentMetadataQuery(DocumentMetadataQueryRequest queryRequest) throws
BaseException;
```

A lekérdezés - sikeres hívás esetén - DocumentMetadataQueryResponse-val térnek vissza, mely tartalmazza a megadott paraméterekre illeszkedő dokumentumok metaadatait, valamint a lapozhatósági adatokat: az összes elem számát, az összes oldal számát, az oldal számát, amelyekhez visszaadtuk a dokumentum metaadatait, valamint, hogy hány elemet adtunk vissza a lekérdezésben.

Paraméterként a szűrési feltételeket, illetve a második esetben a lapozási paramétereket is megadhatjuk. Ha ezt nem tesszük, akkor az első oldal 15 elemét kapjuk vissza a válaszban. Szűrési paraméternek megadható a dokumentum neve, formátuma, a dokumentum tárolási metódusa, a sablon azonosítója, amelyből a dokumentumot generáltuk, a dokumentum tárolójának az azonosítója és a dokumentum státusza.

Lehetőség van arra is, hogy a korábban kigenerált dokumentumot kérdezzük le:

```
public GeneratedDocumentDto getDocumentContent(String documentId) throws
BaseException;
```

A válaszban - sikeres hívás esetén - visszaadjuk a generált dokumentumot, mint stream-et, a fájl nevét, és a HTTP státusz kódot.

Paraméterként a dokumentum egyedi azonosítóját kell megadni.

A kliensnek van pár további beállítási lehetősége is, melyekkel a generálást tudjuk befolyásolni:

```
setGeneratorEngineType()
```

```
setTemplateEngineType()
```

```
setResponseFormat()
```

```
setDocumentStorageMethodType()
```

```
setDigitalSigningType()
```

A `setGeneratorEngineType()` segítségével az output generáláshoz használható engine-t állíthatjuk be, mely jelenleg az alábbiak lehetnek:

- `PDF_BOX` — Apache PdfBox használata
- `SAXON` — Saxon HE használata

- `NONE`

A `setTemplateEngineType()` segítségével template 'típusát' állíthatjuk be, mely jelenleg az alábbiak lehetnek:

- `HANDLEBARS` — Handlebars használata
- `NONE`

A `setResponseFormat()` segítségével a válaszformátumot állíthatjuk be.

- `PDF`
- `STRING`

A `setDocumentStorageMethodType()` segítségével a dokumentum tárolási módját állíthatjuk be.

- `NONE`
- `DATABASE`

A `setDigitalSigningType(digitalSigningType)` segítségével tudjuk vezérelni, hogy a generált PDF dokumentumra kerüljön-e digitális aláírás. A digitalSigning további három paramétert vár, melyek az alábbiak:

- signatureName - az aláírás 'neve' (opcionális)
- signatureReason - milyen célból írta alá (opcionális)
- keyAlias - a keystore-ban lévő kulcs azonosítója, amivel az aláíró kulcsot tudjuk azonosítani (opcionális, de célszerű megadni, mert alapértelmezetten a `test` nevű kulcsot keresi)

Ha nem állítunk be semmit, alapértelmezetten a `PDF_BOX` + `HANDLEBARS` + `PDF` + `NONE` értékek vannak beállítva, digitális aláírás nélkül.

A `postDocumentGenerateEntityBody()` metódusok az alábbi REST végpontot hívják meg a modulban:

```
POST /internal/dookug/document/generate/inline
```

A kliens a kérésben megküldi:

- ContextType-ot
- a kapott TemplateType listát
- a kapott ParameterType listát
- a GeneratorSetup objektumot, amit a kliens `set` metódusaival tudunk kontrollálni.

Ha a kérés megfelelő, akkor visszakapjuk a generált objektumot.

NOTE	A Multipart és StoredTemplate kliensek esetében ugyanígy járunk el, azok csak a REST API hívásokban különböznek.

A `postDocumentMetadataQuery()` metódusok az alábbi REST végpontot hívják meg a modulban:

```
POST /internal/dookug/document/storedTemplate/metadata/query
```

A kliens a kérésben megküldi:

- ContextType-ot
- a kapott szűrési feltételeket
- a lapozási paramétereket
- a sorrendezési beállításokat

Ha a kérés megfelelő, akkor visszakapjuk a kérésben beküldött paraméterekre illeszkedő dokumentum metaadatokat.

Példa a kliens használatához:

```
 @Inject
 private DookugClient dookugClient;
 ...
 //template objektum
 TemplateType template = new
TemplateType().withTemplateName("main").withTemplateContent("DookuG client simple test
with prameters first: [{{first}}], second:
[{{second}}]".getBytes(StandardCharsets.UTF_8));

 //paraméterek
 ParameterType parameter1 = new ParameterType().withKey("first").withValue("első");
 ParameterType parameter2 = new
ParameterType().withKey("second").withValue("í189öüóőúűáé-.,<>#&@{};*¤ß$");
 ...
 client.setResponseFormatType(ResponseFormatType.STRING);
 client.setGeneratorEngineType(GeneratorEngineType.NONE);
 GeneratedDocumentDto response =
dookugClient.postDocumentGenerateEntityBody(List.of(template),
List.of(parameter1,parameter2));
```

vagy hasonlóan dokumentum generálása, de PDF dokumentum formátummal, multipart inputtal, és a metaadatot adjuk vissza

```
 @Inject
 private DookugClient dookugMultipartClient;
 ...
 //template byte tömbként
 byte[] template = "DookuG client simple test with prameters first: [{{first}}],
second: [{{second}}]".getBytes(StandardCharsets.UTF_8);


 //paraméterek
 ParameterType parameter1 = new ParameterType().withKey("first").withValue("első");
```

```
 ParameterType parameter2 = new
ParameterType().withKey("second").withValue("í189öüóőúűáé-.,<>#&@{};*¤ß$");
 ...
 client.setResponseFormatType(ResponseFormatType.PDF); //ez a default
 client.setGeneratorEngineType(GeneratorEngineType.PDF_BOX); //ez a default
 client.setTemplateEngineType(GeneratorEngineType.HANDLEBARS); //ez a default
 DocumentMetadataResponse response =
dookugMultipartClient.postDocumentGenerateMultipartMetadata(new
ByteArrayInputStream(template), List.of(parameter1, parameter2));
```

3.3.2. Saxon(-HE) engine használata a kliensben

Saxon motor használatához szükséges egy XSLT template, ami alapján egy XML-ből PDF fájlt generálhatunk. (Ebben az esetben csak PDF lehet a kimenet) Szükséges egy fop-config.xml fájlt is átadni a requestben, amivel például a fontok használatát tudjuk szabályozni.

```
<?xml version="1.0" encoding="UTF-8"?>
<fop version="1.0">
    <renderers>
        <renderer mime="application/pdf">
            <fonts>
                <!-- TTF fonts -->
                <font kerning="yes" embed-
url="/home/icellmobilsoft/fonts/Roboto/Roboto-Regular.ttf">①
                    <font-triplet name="Roboto" style="normal" weight="normal" />
                </font>
                <font kerning="yes" embed-
url="/home/icellmobilsoft/fonts/Roboto/Roboto-Bold.ttf">
                    <font-triplet name="Roboto" style="normal" weight="bold" />
                </font>
            </fonts>
        </renderer>
    </renderers>
</fop>
```

① Itt tudjuk megadni, hogy a fájlrendszerben hol találhatóak meg a használt betűtípusok.

HANDLEBARS szintén használható a SAXON használatakor, ilyenkor a szokásos {{VARIABLE}} változókba helyettesíthetjük be a kívánt szövegrészeket, valamint készíthetünk egymásba ágyazott template-eket is (itt legfőképp erre lehet használni). Ami változás a többi enginehez képest, hogy itt meg kell adni a generatorSetup osztályban az xmlDataToTransform mezőben a transzformálni kívánt XML fájlt, azaz ebben az esetben az XML fájl nem a template lesz, hanem az adatforrás.

Tehát a generatorSetupban SAXON esetén a többi mező mellett kötelezően megadandó:

- XML: mint adatforrás
- XSLT: mint template
- fopConfig: transzformátor konfiguráció

3.3.3. Hibakezelés

A kliens csak BaseExceptiont adhat vissza, de ha az API hívásban egy RestClientResponseException-t kap vissza, akkor az abban lévő becsomagolt BaseException-t fogja visszaadni!

4. DOCUMENT service

- Main: `dookug-document/dookug-document-service`

4.1. Dokumentum generálása

A modul több lehetőséget is biztosít dokumentum generálására. Mindegyik lehetőségnél küldenünk kell, hogy a sablon-t, illetve a dokumentumot milyen engine dolgozza fel, milyen formátumban szeretnénk megkapni a fájlt, illetve milyen tárolási metódussal mentsük el a legenerált dokumentumot. Ha a generált dokumentumunk formátumának nem STRING értéket adunk meg, de nem adunk meg engine-t, amely feldolgozhatja a dokumentumot, illetve ha STRING formátumot szeretnénk, de engine-nek megadtuk a PDF_BOX-t, akkor INVALID_INPUT hibát adunk vissza. A dokumentum generárálásához, ha a kiinduló sablon alapján szükséges, paramétereket is meg kell adni. Ennek megadására több lehetőség is van, az egyik, hogy kulcs-érték párokban küldjük. A másik esetben egy json szerkezetett várunk el, amit base64binary formában kell küldeni a kérésben.

4.1.1. Dokumentum generálása request body alapján

A generálás során egy request body-ban beküldjük a kiinduló sablont, illetve a generáláshoz tartozó adatokat: sablont és a dokumentumot feldolgozó engine-t, a válasz formátumot, illetve a dokumentum tárolási metódusát.

Minta dokumentum generálás request - json paraméterekkel base64binary formában (a jobb olvashatóság érdekében a sablonok tartalmának és a paraméter értékei le lettek vágva):

```
Request method: POST
Request URI:    http://localhost:8082/internal/dookug/document/generate/inline
Headers:        Accept=application/json
                Content-Type=application/json
```

```
{
  "context": {
    "requestId": "431AZYLPPS6LJE01",
    "timestamp": "2023-02-22T09:12:23.533Z"
  },
  "generatorSetup": {
    "generatorEngine": "PDF_BOX",
    "templateEngine": "HANDLEBARS",
    "parametersData": "ewogICAgIklOU1RJVFVURV9OQU1FIjogIlNpw7Nmb2tpIGvDs3Jow...",
    "responseFormat": "PDF",
    "documentStorageMethod": "DATABASE"
```

```
  },
  "templates": [
    {
      "templateName": "main_template",
      "templateContent": "PCFET0NUWVBFIGh0bWw+CjxodG1sPgo8aGVhZD4KCjxzdHls...",
      "initial": true
    },
    {
      "templateName": "head_template",
      "templateContent": "PHRhYmxlIHN0eWxlPSJoZWlnaHQ6IDE2MXB4OyB3aWR0aDog...",
      "initial": false
    }
  ]
}
```


A válaszban a generált dokumentumot kapjuk vissza, valamint HTTP header-ben a fájl nevét. Lehetőség van arra, hogy ne a dokumentumot kapjuk meg a válaszban, hanem az azt leíró metaadatokat. Ebben az esetben a request URI, és a HTTP header tér el a kérés beküldésekor.

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/generate/inline/metadata
Headers:        Accept=application/json
                Content-Type=application/json
```

4.1.2. Dokumentum generálása multipart form alapján

A generáláshoz be kell küldjük a kiinduló sablont, illetve a generáláshoz tartozó adatokat. Ez utóbbi megegyezik a request body alapján történő dokumentum generálásnál beküldött adatokkal.

Minta dokumentum generálás multipart form request kulcs-érték paraméterekkel:

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/generate/inline/multipart
Headers:        Accept=application/octet-stream
                Content-Type=multipart/form-data

Content-Disposition: form-data; name="REQUEST"
Content-Type: application/json
{
    "context": {
      "requestId": "42ZZBR3LKJZ9IT0X",
      "timestamp": "2023-02-21T12:57:52.113Z"
    },
    "generatorSetup": {
      "generatorEngine": "PDF_BOX",
      "templateEngine": "HANDLEBARS",
```

```
"parameters":[{"key":"first","value":"első"},{"key":"second","value":"í123456789öüóőúű
áé-.,<>#&@{};*¤ß$"}],
      "responseFormat": "PDF",
      "documentStorageMethod": "DATABASE"
    }
}
--2b17c723-ec5b-4c99-8dcf-c6d8972c4564
Content-Disposition: form-data; name="TEMPLATE"
Content-Type: application/octet-stream

DookuG simple test with prameters first: [{{first}}], second: [{{second}}]
```

A válaszban a generált dokumentumot kapjuk vissza, valamint HTTP header-ben a fájl nevét. Lehetőség van arra is, hogy ne a dokumentumot kapjuk meg, hanem az azt leíró metaadatokat. Ebben az esetben a request URI, és a HTTP header tér el a kérés beküldésekor.

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/generate/inline/multipart/metadata
Headers:        Accept=application/json
                Content-Type=multipart/form-data
```

4.1.3. Dokumentum generálása mentett sablon alapján

A generáláshoz be kell küldenünk a sablon nevét, amiből generáltatjuk a dokumentumot, valamint a generáláshoz tartozó adatokat. Ez utóbbi megegyezik a request body alapján történő dokumentum generálásnál beküldött adatokkal, kiegészülve azzal, hogy a sablont milyen tárban tároljuk.

Példa dokumentum generálása mentett sablon alapján request - json paraméterekkel base64binary formában (a jobb olvashatóság érdekében a paraméter értéke le lett vágva):

```
Request method: POST
Request URI:    http://localhost:8082/internal/dookug/document/generate/storedTemplate
Headers:        Accept=application/json
                Content-Type=application/json
```

```
{
  "context": {
    "requestId": "431BIFEOUJ0ODU01",
    "timestamp": "2023-02-22T09:26:45.121Z"
  },
  "generatorSetup": {
    "templateStorageMethod": "DATABASE",
    "template": {
      "templateName": "DEV_TEMPLATE_HANDLEBARS",
```

```
      "templateLanguage": "HU",
      "validityDate": "2023-02-22T09:26:45.130074Z"
    },
    "generatorEngine": "PDF_BOX",
    "templateEngine": "HANDLEBARS",
    "parametersData": "ewogICJ0aXRsZSI6ICJwZWxkYSBjaW0iLAogICJjdXJyZW50WWV...",
    "responseFormat": "PDF",
    "documentStorageMethod": "DATABASE"
  }
}
```


A válaszban a generált dokumentumot kapjuk vissza, valamint HTTP header-ben a fájl nevét. Lehetőség van arra, hogy ne a dokumentumot kapjuk meg a válaszban, hanem az azt leíró metaadatokat. Ebben az esetben a request URI, és a HTTP header tér el a kérés beküldésekor.

Az adatbázisban tárolt template kulcsa a templateName és templateLanguage értékéből áll össze.

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/generate/storedTemplate/metadata
Headers:        Accept=application/json
                Content-Type=application/json
```

A dokumentum mentése a `documentStorageMethod` paraméter megadásától függ. Két értéket vehet fel jelenleg: `NONE` és `DATABASE` Amennyiben a `NONE` értéket adjuk meg, akkor a dokumentumot nem mentjük el, így a későbbiekben nem is lehet lekérdezni. A `DATABASE` esetében a generált dokumentumot adatbázis táblába mentjük el, és a későbbi lekérdezéskor, onnan olvassuk ki. Az adatbázisba elmentjük még a dokumentumhoz tartozó egyéb adatokat is:

- a kiinduló sablon azonosítóját - abban az esetben, ha nincs elmentve a sablon ezt a paramétert nem töltjük
- a generált fájl nevét - Három részből generáljuk a fájlnevet: a dokumentum egyedi azonosítója, a kiinduló sablon neve, és a generálás időpontja long értéke
- a fájl formátuma
- a dokumentum státusza - DONE, FAILED, PENDING, SYNCING
- a dokumentumhoz tartozó paraméterek
- a dokumentum tárolási formája - `DATABASE` esetén ezt a mezőt is DATABASE értékkel töltjük

---

A generálás során bármilyen formában adjuk meg a kiinduló sablont a válaszban vagy a generált fájlt kapjuk vissza, vagy a dokumentumot leíró metaadatokat, DocumentMetadataResponse típusú objektumként.

Példa DocumentMetadataResponse:

```
{
    "context": {
      "requestId": "42ZZBQ5K7W43FI6W",
      "timestamp": "2023-02-21T12:57:50.888Z"
    },
    "funcCode": "OK",
    "metadata": {
      "documentId": "42ZZBQ3ISCXWVO6V",
      "storageMethod": "DATABASE",
      "filename": "filename.pdf",
      "format": "PDF",
      "status" : "DONE"
    }
}
```

4.2. Dokumentum metaadatainak lekérdezése

A dokumentum metaadatok lekérdezésének célja, hogy a megadott szűrési feltételeknek megfelelő dokumentum adatokat visszaadjuk.

A végpont biztosítja a lapozhatóságot, vagyis az adatokat több oldalon keresztül kaphatjuk meg. Ehhez a kérésben beküldhetjük, hogy melyik oldal adatait szeretnénk visszakapni, illetve az oldalról mennyi elemet. Ennek megfelelően a válaszban visszaadjuk, hogy összesen hány elem található, illetve, hogy az elemek mennyi oldalon férnek el. Amennyiben nem küldjük ezt a paramétert a végpont alapértelmezetten az első 15 elemet fogja visszaadni.

A végponton a következő szűrési feltételek adhatóak meg:

- templateId - a dokumentum generáláshoz használt sablon azonosítója
- status - a dokumentum státusza
- format - a dokumentum fájl formátuma
- storageMethod - a dokumentum tárolási formája
- storageId - a dokumentum tárolójának egyedi azonosítója
- filename - a dokumentum fájl neve

A rendezési paraméter lehet:

- filename
- documentStorageMethod
- format
- status

A rendezési paraméterekhez meg lehet adni külön-külön, hogy csökkenő, illetve növekvő rendezést szeretnénk. Az előbb említett rendezéseken kívül, van egy alapértelmezett rendezés a dokumentum azonosítójára.

Minta DocumentMetadataQueryRequest:

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/storedTemplate/metadata/query
Headers:        Accept=application/json
                Content-Type=application/json; charset=UTF-8
```

```
{
    "context": {
      "requestId": "43183LDKQNC2R702",
      "timestamp": "2023-02-22T09:15:14.168Z"
    },
    "paginationParams": {
      "rows": 10,
      "page": 1
    },
    "queryParams": {
      "status": "DONE",
      "storageMethod": "DATABASE",
      "filename": "filename.pdf",
      "format": "PDF",
      "templateId" : "MAIN_TEMPLATE"
    }
}
```

Ha a kérésben beküldött paraméterek alapján található dokumentum metaadat, akkor a válaszban egy maximum 100 elemű listában visszakapjuk az adatokat.

Minta DocumentMetadataQueryResponse:

```
{
    "context": {
      "requestId": "43183LDKQNC2R702",
      "timestamp": "2023-02-22T09:15:14.168Z"
    },
    "funcCode": "OK",
    "rowList": {
      "documentId": "43183KXXW2KCI206",
      "storageMethod": "DATABASE",
      "filename": "filename.pdf",
      "format": "PDF",
      "status" : "DONE"
    }
  }
```

4.3. Dokumentum lekérdezése

A végpont célja, hogy a már legenerált és elmentett dokumentumot a megadott azonosító alapján visszaadja.

Minta dokumentum lekérdezés request:

```
Request method: GET
Request URI:
http://localhost:8082/internal/dookug/document/content/43183KXXW2KCI206 ①
Headers:        Content-Type=application/octet-stream
```

① A generált dokumentum azonosítója

Amennyiben a beküldött azonosítóhoz nem található dokumentum, akkor ENTITY_NOT_FOUND hibát adunk vissza.

A válaszban - létező dokumentum azonosító esetén - visszaküldjük a generált dokumentumot, és HTTP header-ben a fájl nevét is.

4.4. Template cache

Az adatbázisban tárolt TEMPLATE adatokat cache-eljük (GUAVA). A cache-k meghatározott ideig élnek, új lekéréskor az idő újra indul.

Konfigurációs paraméterek:

DOOKUG_SERVICE_CACHE_TEMPLATE_TTL paraméterrel adhatjuk meg percben meddig tartsuk a cachben a templateket, alapértelmezetten 60 perc.

DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLESTATISTIC paraméterrel adhatjuk meg, hogy szükséges-e a metrikák generálása , alapértelmezetten false, nem képződnek metrikák.

Az alábbihoz hasonló metrika generálódik:

```
# TYPE application_cache_hit_count gauge
application_cache_hit_count{name="template"} 0.0
# TYPE application_cache_miss_count gauge
application_cache_miss_count{name="template"} 1.0
# TYPE application_cache_size gauge
application_cache_size{name="template"} 1.0
```

4.5. Saját betűtípusok használata

A leírás az Apache PDFBox engine-hez készült és a saját betűtípusok használatáról szól.

NOTE	Az engine limitációja, hogy csak TTF típusú betűtípusokat tud beágyazni, valamint

> ha használni szeretnénk a weight,styles,variants beállításokat, akkor érdemes tudni, hogy ezeket a pdfbox nem tudja mindig megfelelően emulálni, hacsak nem töltjük be a font különböző variációit is.

A docker-compose fájlban a `fonts` könyvtárat hozzáadjuk `/home/icellmobilsoft/fonts` -ként az image-ünkhöz, így az ebben a könyvtárban lévő fontokat lokálisan el tudjuk érni a template-ekből.

Két lehetőségünk van arra, hogy saját betűtípusokat tudjunk használni.

1. közvetlenül a CSS-ben töltjük be a fontot
2. programozottan töltjük be, majd ezután a CSS-ben meghivatkozzuk a referenciát (`font-family`) használatával

Az első opcióban a CSS-ben a `@font-face` at-rule-on belül az `src: url()` leíróban adhatjuk meg a betűtípus elérhetőségét.

Ebben az esetben internet elérésre lesz szükségünk a generáláskor, ami nem feltétlenül lehet adott minden esetben, mert tilthatja a vállalat policy-je, hogy a modul külső url-eket érhessen el.

Ez kivédhető vagy úgy, hogy belső URL-en keresztül lehet elérni a használni kívánt fontokat, vagy a modulban előre feltöltünk néhány gyakran használt betűtípust és a lokális elérhetőségét adjuk meg file:// url-lel meghivatkozva. Ha a modul tartalmazni fog előre betöltött betűtípusokat, akkor célszerű ezeket a dokumentációban jelezni, vagy egy végponton visszaadni az információkat.

TEMPLATE:

```
<style>
    @font-face {
        font-family: 'Cairo';
        font-style: normal;
        src: url(file:/home/icellmobilsoft/fonts/Cairo/Cairo-Regular.ttf); ①
    }

    @font-face {
        font-family: 'IndieFlower';
        font-style: normal;
        src: url(file:fonts/IndieFlower/IndieFlower-Regular.ttf); ②
    }
</style>
```

① a `/home/icellmobilsoft/fonts` könyvtárban található Cairo-Regular.ttf betöltése.

② Itt szintén a fenti könyvtárból töltjük be a fontot. A `/home/icellmobilsoft` -ot tekinti a server root-ként, ezért működik a relativ útvonal megadása.

TIP

Google fontok használata

```
<style>
    @import
```

```
url('https://fonts.googleapis.com/css?family=Quicksand&amp;display=swap');
①

    .quicksand { ②
      font-family: 'Quicksand', Arial;
      font-weight: 700;
      font-style: normal;
      font-size: 38px;
      line-height: 1.15;
      letter-spacing: -.02em;
      color: rgba(0, 0, 0, 0.8);
      -webkit-font-smoothing: antialiased;
    }
</style>
...
<p class="quicksand">Google Quicksand Font úőüöóéáyí</p> ③
```

① Quicksand betűtípus importja a Google Fonts -ról

② CSS class beállítása a font használatához

③ Szöveg kiírása a beállított betűtípussal

Ha fájlrendszerből programozottan szeretnénk betölteni a betűtípusokat, akkor a `builder.useFonts` metódusát tudjuk használni a renderkor, melyekre a CSS-ben hivatkozni tudunk.

JAVA:

```
builder.useFont(new File("fonts/NotoSansThai/NotoSansThai-Regular.ttf"),
"notosansthai-regular"); ①
```

① NotoSansThai-Regular.ttf betöltése

TEMPLATE:

```
<style>
    @font-face {
        font-family: 'notosansthai-regular'; ①
        font-style: normal;
        -fs-font-subset: complete-font; ②
    }
</style>
```

① fentebb betöltött betűtípus használata

② ez csak példaként van itt, a beállítást nem igazán szükséges használni, mert így a teljes font beágyazásra kerül, viszont alapértelmezetten csak a subset, ami a helyes működés.

Ennek van egy olyan hátránya, hogy mindegyik fontot betölti a renderkor, azokat is, amiket nem használunk a dokumentum template-ben.

NOTE A PDFBox Fonts Wiki itt elérhető

4.6. PDF-ek digitális aláírása

A leírás az Apache PDFBox engine-hez készült és a digitális aláíráshoz egy self signed certificate-et használ, melyet a docker-compose fájl segítségével adunk hozzá az image-hez `/home/icellmobilsoft/keys/keystore.p12` fájlként.

A keystore az `/etc/docker-compose/keys` mappában található és 2023. november 7-én az alábbi paranccsal lett létrehozva:

```
keytool -genkeypair -storepass 123456 -storetype pkcs12 -alias test -validity 10958 -v
-keyalg RSA -keystore keystore.p12
```

WARNING Ez egyben azt is jelenti, hogy 2053. november 7-én le fog járni.

4.6.1. Konfiguráció

Az aláírást az API hívásokban a request GeneratorSetup -jában tudjuk kérni. Itt meg tudjuk adni a `signatureName`-et, `signatureReason`-t, valamint az private key alias-t (`keyAlias`) arra az esetre, ha a keystore fájl több private kulcsot is tartalmazna.

```
...
<ns2:generatorSetup>
    <ns2:generatorEngine>PDF_BOX</ns2:generatorEngine>
    <ns2:templateEngine>NONE</ns2:templateEngine>
    <ns2:responseFormat>PDF</ns2:responseFormat>
    <ns2:addDigitalSignature>
        <ns2:signatureName>TestSuite name</ns2:signatureName>
        <ns2:signatureReason>TestSuite teszt reason</ns2:signatureReason>
        <ns2:signatureProfile>sampleProfile</ns2:signatureProfile>
    </ns2:addDigitalSignature>
        ...
```

Az aláírást a következő konfigurációs kulcsokkal tudjuk testreszabni:

```
dookug:
  service:
    engine:
      pdf:
        digitalsign:
          signature:
            sampleProfile: ①
              name: Példa kft. ②
              reason: Hitelesített ②
              keystore: /home/icellmobilsoft/keys/keystore.p12 ③
```

```
                keystorePass: 123456 ④
                keystoreType: PKCS12 ④
                keyAlias: kulcs_teszt ⑤
```

① a signatureProfile neve

② a `name` és a `reason` default értékek arra vannak, ha a request nem tartalmazná ezen értékeket.

③ a `keystore` kötelezően megadandó, az aláíró kulcsok gyűjtőhelyét tartalmazza.

④ a `keystorePass`, `keystoreType` a keystore fájlhoz tartozó jelszót, valamint a keystore típusát adja meg, ezek szintén kötelezően megadandó értékek.

⑤ a keystore-on belül a privát kulcs azonosítója

5. Telepítés, üzembe helyezés

A Dookug modul a szolgáltatást felhasználni kívánó projekt(ek) környezeteiben elérhető kell legyen. Ehhez a service egy-egy példánya - infrastrukturális szükségleteivel együtt - minden egyes (fejlesztői/teszt/éles) környezetre telepítendő és konfigurálandó.

5.1. DookuG Service Konfiguráció

Interface

```
DOOKUG_SERVICE_INTERFACE_PARAMETERSDATA_GZIPPED=true
```

PDF aláírás

```
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_NAME="Eredeti dokumentum"
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_REASON="Igazolva I-Cell
Mobilsoft Zrt. által"
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_KEYSTORE=/home/icellmobilsoft/
keys/keystore.p12
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_KEYSTOREPASS=123456
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_KEYSTORETYPE=PKCS12
```

Saxon

```
DOOKUG_SERVICE_ENGINE_SAXON_FOPCONFIG=/home/icellmobilsoft/fop-config/fop-config.xml
DOOKUG_SERVICE_ENGINE_SAXON_XSLT_LANGUAGE_VARIABLE=lang
DOOKUG_SERVICE_ENGINE_SAXON_XSLT_LANGUAGE_DEFAULT=HU
```

Handlebars

```
DOOKUG_SERVICE_ENGINE_HANDLEBARS_HELPER_JAVASCRIPT_DIRECTORY=/home/icellmobilsoft/hand
lebars/helper/js
DOOKUG_SERVICE_ENGINE_HANDLEBARS_ESCAPINGSTRATEGY=HTML_ENTITY
```

Memory cache

```
DOOKUG_SERVICE_CACHE_TEMPLATE_TTL Alapértelmezetten 60 perc
DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLESTATISTIC Alapértelmezetten false
```

Cache

```
DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLED=true ①
DOOKUG_SERVICE_CACHE_TEMPLATE_TTL=60 ②
DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLESTATISTIC=false ③
```

① template cache-elés ki (false) vagy bekapcsolása (true), alapértelmezetten: true

② template cache-ben lévő értékek TTLje percben megadva, alapértelmezetten 60

③ template cache statisztika nyújtásának ki (false) vagy bekapcsolása (true), alapértelmezetten: false

5.1.1. Kubernetes deployment

- Ajánlott konfiguráció

Parameter	Value	Description
JAVA_OPTS	-Xms2000m -Xmx2000m -XX:MetaspaceSize=96M -XX:MaxMetaspaceSize=256m -XX:+PrintCommandLineFlags -XX:+UseG1GC	2G memóriában maximalizáljuk a modul heap szükségletét, UseG1GC algoritmus használatával.

Requests/limits beállítások

```
spec:
  containers:
    resources:
      limits:
        cpu: "3"
        memory: 3G
      requests:
        cpu: "3"
        memory: 3G
```

5.1.2. Health - startup/liveness/readiness

A service támogatja a k8s probe-ok használatát.

- Beépített indítási ellenőrzések: database

http://localhost:9990/health/started

```
{
  "status": "UP",
  "checks": [
    {
      "name": "started-deployment.ROOT.war",
      "status": "UP"
    }
  ]
}
```

http://localhost:9990/health/live

```
{
  "status": "UP",
  "checks": [
    {
      "name": "empty-liveness-checks",
      "status": "UP"
    }
  ]
}
```

http://localhost:9990/health/ready

```
{
  "status": "UP",
  "checks": [
    {
      "name": "deployments-status",
      "status": "UP",
      "data": {
        "ROOT.war": "OK"
      }
    },
    {
      "name": "server-state",
      "status": "UP",
      "data": {
        "value": "running"
      }
    },
    {
      "name": "boot-errors",
      "status": "UP"
    },
    {
      "name": "ready-deployment.ROOT.war",
      "status": "UP"
```

```
    }
  ]
}
```

6. Kiegészítő információk

6.1. Helpers - Template fájlokban használható helperek

6.1.1. Built-in helperek

A Handlebars biztosít beépített helpereket, amelyeknek a dokumentációja itt található.

6.1.2. Custom helperek

Lehetőség van a modulban biztosított általános helpereket használni. A példákban a ... részek renderelődnek ki amennyiben a kifejezés igazra értékelődik ki.

Használatuk

A helperek használata során a helper kulcsszava után kell megadni az adott helperhez tartozó paramétereket. Inline generálás és adatbázisban tárolt template alapú generálás során is felhasználhatóak a helperek. A paraméterek lehetnek beégetett értékek vagy jöhetnek a dokumentumgenerálási hívás során JSON formátumban.

A helperek kombinálhatók egymással.

Token	Magyarázat	Használat	Ettől a verziótól
equals	Két elem értékét hasonlítja össze, akkor történik renderelés ha igaz a kiértékelés eredménye.	`{{#if (equals yourField 'white')}} ... {{/if}}`	0.1.0
before	Akkor történik renderelés, ha az átadott paraméter korábbi időpont mint a biztosított érték. Utóbbi szintén lehet paraméter.	`{{#if (before yourDate '2023-08-13T05:40:55Z')}} ... {{/if}}` `{{#if (before yourDate checkDate)}} ... {{/if}}`	0.1.0

between	Akkor történik renderelés, ha az átadott paraméter a két biztosított dátum érték közé esik. Utóbbiak szintén lehetnek paraméterek.	`{{#if (between yourDate '2023-08-13T05:40:55Z' '2023-08-15T05:40:55Z')}} ... {{/if}}` `{{#if (between yourDate startDate endDate)}} ... {{/if}}`	0.1.0

dateMinusMinutes	Adott dátumból megadott percek kivonása.	`{{dateMinusMinutes '2023-08-13T05:40:55Z' 60}}  // '2023-08-13T04:40:55Z' lesz a kiértékelés eredménye`
		`{{dateMinusMinutes yourDate 60}}`
		`// A következő két példa ugyanezzel a dátummal értékelődik ki`
		`{{dateMinusMinutes yourDate minutesToSubtract}}`
			0.1.0

datePlusMinutes	A dateMinusMinutes párja, hozzáadunk perceket a kapott dátumhoz.	`{{datePlusMinutes '2023-08-13T05:40:55Z' 60}} - '2023-08-13T06:40:55Z' lesz a kiértékelés eredménye`
		`{{datePlusMinutes yourDate 60}}`
		`{{datePlusMinutes yourDate minutesToAdd}}`
			0.1.0

declare	Új változó létrehozása és azonnali renderelése. Az így létrehozott változók globálisak, függetlenül attól hogy hol hozták létre a template-ekben.	`{{declare 'myColor' 'white'}}`
		`// azonnal ki is rendereljük a 'white' szót`
		`...`
		`{{#if myColor}} {{myColor}} {{/if}}`
		`// 'white' lesz renderelve`
			0.1.0

declareVoid	Hasonlóan működik mint a declare helper, azonban ez nem rendereli ki azonnal a kapott értéket. A declare kiváltható lenne a declareVoid-dal létrehozott változóval majd azonnali meghívással.	`{{declareVoid 'myColor' 'white'}}`
		`...`
		`{{#if myColor}} {{myColor}} {{/if}}`
		`// 'white' lesz renderelve`
		`declare helper kiváltása:`
		`{{declareVoid 'myColor' 'white'}}`
		`{{myColor}}  // 'white' lesz renderelve azonnal a létrehozás után`
			0.1.0

formatDate	A kapott, Java 8 által elfogadott dátum formátumú paraméter átalakítása megadott pattern szerint. A pattern csak dátum formátumot fogad el, ellenkező esetben hibát dob.	`yourDate = '2024-01-09';`
		
		`{{formatDate '2023-08-13' 'yyyy.MM.dd'}} // '2023.08.13'`
		`{{formatDate '2023-08-13' 'MM.dd.yyyy'}} // '08.13.2023'`
		`{{formatDate yourDate 'yyyy/MM/dd'}} // '2024/01/09'`
			0.5.0

formatDateTime	A kapott, Java 8 által elfogadott dátum és idő formátumú paraméter átalakítása megadott pattern szerint. A helper kezeli a teljes ISO dátum formátumú inputot, abból képes pattern segítségével megfelelő output előállítására. A helper képes időzónát is kezelni, amennyiben az output pattern után megadunk egy Java által elfogadott Zone ID-t, a bemenő időpontot átalakítja az időzóna szerinti output-tá.	`yourDate = '2024-01-09T15:30:04Z'`
		`yourDateTimeFormat = 'MM.dd-HH:mm'`
		`yourDateFormat = 'yyyy/MM.dd'`
		`yourTimeFormat = 'hh:mm:ss a'`
		
		`{{formatDateTime '2023-08-13T05:40:55Z' 'yyyy-MM-dd HH:mm:ss'}} // '2023-08-13 06:40:55'`
		`{{formatDateTime yourDate 'yyyy-MM-dd HH:mm:ss'}}`
		`// '2024-01-09 15:30:04'`
		`{{formatDateTime yourDate yourDateTimeFormat}}`
		`// '01.09-15:30'`
		`{{formatDateTime yourDate yourDateFormat}}`
		`// '2024/01.09'`
		`{{formatDateTime yourDate yourTimeFormat}}`
		`// '03:30:04 PM'`
		`{{formatDateTime '2023-08-13T05:40:55Z' 'yyyy-MM-dd HH:mm:ss' 'CET'}} // '2023-08-13 08:40:55'`
			0.5.0

formatTime	A kapott, Java 8 által elfogadott idő formátumú paraméter átalakítása megadott pattern szerint. A pattern csak idő formátumot fogad el, ellenkező esetben hibát dob.	`yourTime = '15:30:55Z';`
		
		`{{formatTime '15:30:55Z' 'HH:mm:ss'}} // '15:30:55'`
		`{{formatTime yourTime 'h:mm A'}}`
		`// '3:30 PM'`
			0.5.0

formatNumber	Számok formázására használt helper, JAVA-s számformázási konvenciót követve működik	`number = 1234.567;`
		`percentage = 0.4567`
		
		`{{formatNumber number '#'}}`
		`// "1235", kerekített egész szám`
		`{{formatNumber number '0.00'}}`
		`// "1234.57", 2 tizedesre kerekített szám`
		`{{formatNumber number '000000.00'}}`
		`// "001234.57", vezető nullákkal feltöltött szám`
		`{{formatNumber number '#,###.##'}}`
		`// "1,234.57", vesszővel elválaszott, hármasával csoportosított érték`
		`{{formatNumber number '$#,##0.00'}}`
		`// "$1,234.57", pénznem kifejezése`
		`{{formatNumber percentage '0.00%'}}`
		`// "45.67%", 0 és 1 közötti érték százalékos kifejezése 0 és 100% között`
		`{{formatNumber number '0.###E0'}}`
		`// "1.235E8", tudományos jelölésű szám`
			0.1.0

and	N érték közti && operátor. Akkor renderelük ha a logikai ÉS operátor igazra értékelődik ki.	`{{#if (and falseValue trueValue notExistingValue)}} YES {{else}} NO {{/if}}  // "NO" érték renderelődik`
		`{{#if (and trueValue trueValue trueValue)}} YES {{/if}}`
		`// "YES" renderelődik mert ugyanaz mindhárom érték és azok igazak`
		`myValue = 'black'`
		`{{#if (and (equals myValue 'black')}} YES {{else}} NO {{/if}}`
		`// "YES" renderelődik ki, a változó és a string ÉS kiértékelése igaz`
		`{{#if (and (equals myValue 'white')}} YES {{else}} NO {{/if}}`
		`// "NO" renderelődik ki, a változó és a string ÉS kiértékelése hamis`
		
		`Kombinált használat:`
		`{{#if (and (equals 'black' 'white') (equals 'white' 'white'))}} YES {{else}} NO {{/if}}  // "NO" renderelődik ki, az első hamisra, a második igazra értékelődik ki`
			0.1.0

or	N érték közötti || operátor. Akkor renderelük ha a logikai VAGY operátor igazra értékelődik ki.	`{{#if (or falseValue trueValue notExistingValue)}} YES {{else}} NO {{/if}}  // "YES" érték renderelődik`
		`{{#if (or falseValue falseValue falseValue)}} YES {{/if}}`
		`// Semmi nem renderelődik ki`
		`myValue = 'white'`
		`{{#if (or myValue 'black')}} YES {{else}} NO {{/if}}`
		`// "YES" renderelődik ki, a változó és a string VAGY kiértékelése igaz`
		
		`Kombinált használat:`
		`{{#if (or (equals 'black' 'white') (equals 'white' 'white'))}} YES {{else}} NO {{/if}}  // "YES" renderelődik ki, hamis VAGY igaz esetén igaz a kiértékelés`
			0.1.0

not	Logikai ! operátor. A kapott paramétert negálja, ha igaz a kiértékelés akkor renderelünk, ha hamis akkor nem.	`{{#if (not falseValue)}} YES {{else}} NO {{/if}}`
		`// YES`
		`{{#if (not existingValue)}} YES {{else}} NO {{/if}}  // NO`
		`{{#if (not (equals 'black' 'white'))}} YES {{else}} NO {{/if}}`
		`// YES`
			0.1.0

in	Azt vizsgálja hogy a kapott legelső paraméter megegyezik-e a soron következő elemek bármelyikével.	`myValue = 'white'`
		`{{#if (in myValue 'black' 'gray')}} YES {{else}} NO {{/if}}`
		`// NO`
		`{{#if (in myValue 'black' 'white' 'gray')}} YES {{else}} NO {{/if}}`
		`// YES`
			0.1.0

math	Alap matematikai műveletek elvégzésére használható helper. Az első paraméter az operátor, a másik két paraméter az operandusok. A használható operátorok listája:
	
	“+”, “-”, “*”, “/”, “%”
	
	Hibás operátor esetén a kiértékelés eredménye: “-1”. Egyéb esetben az operandusokon elvégzett, operátornak megfelelő matematikai művelet.
		`num1 = 5`
		`num2 = 8`
		`num3 = 100`
		`num4 = 20`
		
		`{{math '+' num1 num2}}  // 13`
		`{{math '-' num3 53}}  // 47`
		`{{math '*' num2 num4}}  // 160`
		`{{math '/' num3 num4}}  // 5`
		`{{math '%' num4 num2}}  // 40`
		`{{math 'A' num1 num2}}  // -1`
			0.1.0

További helper függvények

A projektben a következő 3rd party helper függvények használhatók:

- StringHelpers

7. Release notes

7.1. v0.4.0

Projekt kikerült Open source Github-ra.

- maven pom change:
  - `<groupId>hu.icellmobilsoft.dookug.common</groupId>` → `<groupId>hu.icellmobilsoft.dookug</groupId>`

- `<groupId>hu.icellmobilsoft.dookug.document</groupId>` → `<groupId>hu.icellmobilsoft.dookug</groupId>`
- `<groupId>hu.icellmobilsoft.dookug.client</groupId>` → `<groupId>hu.icellmobilsoft.dookug</groupId>`

7.2. v0.5.0

7.2.1. Changes / updates

- Caching of templates in memory https://github.com/i-Cell-Mobilsoft-Open-Source/DookuG-backend/issues/4
- Observability: DB health check added
- TemplateLanguage type has been changed from enum to string (`EN` → `EN` or `en_GB`).
- The jandex.idx file has been removed from dookug-client-jee10. It caused Unsatisfied Dependency Exception during the deployment.

7.2.2. Migration

Replace `TemplateLanguageType.*` occurrences with the string value of the language.

7.3. v0.6.0

7.3.1. Changes / updates

- Documentation fixes (formatting, missing 0.5.0 release notes include)
- Add com.github.jknack.handlebars.helper.StringHelpers build in helper to helpers
- Rename formatDate to formatDateTime
- Create formatDate which handles date format
- Create formatTime which handles time format
- Add IT test for owned helpers
- GET `/system/evict` REST endpoint added
- Template caching has been fixed
- Template caching can be configured with the environment variable named `DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLED` (`true` by default)

7.4. v1.0.0

7.4.1. Changes / updates

- Jakarta EE10 upgrade
- Test fix

- gitHub Workflows - java 17 upgrade